

Connecting Agile UX With Agile Software Development

Highlights

- Basics of agile software engineering (SE) methods
 - Lifecycle aspects
 - Planning
 - Sprints
- Challenges in agile SE from the UX perspective
 - What is needed on the UX side?
 - Problems to anticipate
- A synthesized approach to integrating agile UX
 - Integrating UX into planning
 - Integrating UX into sprints
- Synchronizing the two agile workflows

26.1 INTRODUCTION

In [Chapter 4](#), we introduced the agile UX funnel model, and in all subsequent process chapters we have mostly assumed an agile UX process. Now, we discuss some details about how the agile UX process can connect to an agile software engineering (SE) process in real-world project environments.

Agile SE approaches, now well known and commonly used, are an incremental, iterative, and test-driven means of frequently delivering pieces of useful working software to customers. In the past, agile SE approaches have not taken UX into account. Because traditional UX processes have not fit well within agile SE environments, the UX side has struggled to adjust its methods to fit SE constraints.

At the end of the day, the entire system development team needs an overall approach that includes agile UX while retaining the basics of agile SE approaches. In this chapter, we present a variation of our agile UX process methods that will integrate well with existing agile SE processes.

26.1.1 Agility Is Not (Just) About Being Fast

Lots of people think the point of an agile approach is to be fast, but that thought misses the main point. Being agile means being light on your feet, nimble, and responsive to change during the process. An agile process is also considered fast. Compared to the old waterfall method, agile methods are very fast. Being fast is just a bonus side effect, however, not the essence of agility. Agility is about *how* the process is fast.

Agility is about being fast by not wasting time unnecessarily. It is about being fast by knowing what does not work before investing time and effort pursuing the wrong course. It is about being fast by focusing on the product or system rather than the process. More than anything else, it is about being nimble and responsive to change. A project is not a race; it is an obstacle course or, sometimes, even a labyrinth. You cannot know what might derail you from your course until you encounter it and have to react. In the old, slow, and careful waterfall method, you never got the feedback telling you there is a problem to respond to until the whole project was almost over.

Example: Working Too Fast Can Hurt You in the Marketplace

A real-world, web-based B2B software product company in San Francisco had a well-established customer base for its large complex suite of tools. At some point, they made major revisions to the product design as part of normal growth of functionality and market focus. Operating under at least what they perceived as extreme pressure to get it to the market in what they proudly called “internet time,” they released the new version too fast.

The concept was sound, but the design was not well conceived, and the resulting poor usability led to a very bad user experience. Because their customers had invested heavily in their original product, they had a somewhat captive market. By and large, their users were resilient and grumbled but adapted; however, the new version got bad reviews, and the company’s reputation for user experience was changing for the worse. They were not able to bring in new customer business.

So, they panicked, took it back to the drawing board, and pulled everyone in to work on an improved version, this time applying some principles of UX design. How did the marketplace react? Well, potential new customers did not take the bait. The company’s reputation was already spoiled.

What about the existing customers? The immediate reaction from established customers and users was one of betrayal. They had invested the time and energy in adapting to the bad design, and now the company changed it on them—again.

Although the new design was better, existing users were mostly concerned at this point about a new learning curve blocking their productivity once again. This was a defining case of taking longer to do it right versus taking less time to do it wrong and then taking even longer to fix it. By not using an effective UX process in the first place, the company had managed to alienate both their existing and future customer bases. Therein lies a profound lesson about sacrificing quality for the sake of speed: If you work in internet time, you can also crash and burn in internet time!

26.2 BASICS OF AGILE SOFTWARE ENGINEERING METHODS

Much of what is said here about agile methods in SE goes back to the fundamental work of Beck (2000), an authoritative source on agile SE development methods as embodied in the approach called extreme programming (XP).¹ We have taken words from Beck and other authors and tried to blend them into a summary of the practice. Accurate representations are credited to these authors, and errors in representation are on us.

26.2.1 Goals and Principles of Agile Software Engineering

To establish agreement on early agile software methods, a workshop in Snowbird, Utah, in February 2001, formulated an agile manifesto² from which the following principles emerged:

- Satisfy the customer through early and continuous delivery of valuable software. This is our highest priority.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for the shorter timescale.
- Work together daily throughout the project—business people and developers.

¹There are other approaches to agile SE methods beyond XP, including scrum (Rising & Janoff, 2000), but for convenience we focus on XP.

²<http://agilemanifesto.org/>

- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- Convey information to and within a development team. The most efficient and effective way is face-to-face conversation.
- Let working software be the primary measure of progress.
- Promote sustainable development through agile processes. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Pay continuous attention to technical excellence and good design; they enhance agility.
- Practice simplicity, the art of maximizing the amount of work not done. It is essential.
- Use self-organizing teams, the source of best architectures, requirements, and designs.
- Reflect on how your team can become more effective, then tune and adjust your behavior accordingly.

Also from the manifesto, practitioners of agile SE methods value the following:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

26.2.2 Contrasting With the Waterfall Method

Such general principles can seem like platitudes, but it was not always so obvious. Perhaps the best way to understand the impact of these principles is through the most extreme contrast, a contrast with the old software engineering waterfall lifecycle development methodology (Royce, 1970). The waterfall model (Fig. 26.1) is an arduous process in the sense that it proceeds slowly and deliberately, finishing all of one process activity for the entire

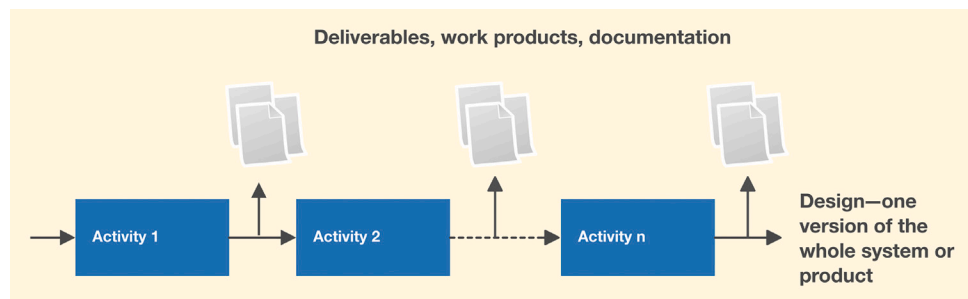


Fig. 26.1
Heavy waterfall process
with deliverables and
documentation after
every process activity.

project before starting the next, and thereby not being responsive to change. By the end, needs and requirements have changed, the product is off course and mostly irrelevant, and resources are wasted.

26.2.2.1 Operating in silos

Major lifecycle activities in the waterfall model were sometimes called silos because they strongly compartmentalized the activities. Each stage of the lifecycle was a complete enterprise in itself, occurring in its own closed environment (silo). In each silo, a team of specialists for that stage worked with its own management and produced its own deliverables. Each stage typically created voluminous documentation.

There was no working version of anything until the very end of the lifecycle, at which point you got one version of the whole system or product, which by then was likely to be out of date. Certainly, any requirements that had been painstakingly developed and documented in the requirements phase would no longer be complete or correct.

26.2.2.2 Characteristics of agile software engineering methods

In agile SE development environments, coding begins very early. Agile SE has a shorter, almost nonexistent requirements engineering phase and far less documentation than that of traditional SE. As typified in XP, agile SE code implementation occurs in small increments and iterations.

Small releases are delivered to the customer after each short iteration or development cycle. In most cases, each small release, although limited in functionality, is intended to be a working version of the evolving system that delivers some useful capability for the customer.

In straight-forward terms, agile SE development methods “describe the problem simply in terms of small, distinct pieces, then implement these pieces in successive iterations” (Constantine, 2002, p. 3). Each piece is tested until it works and is then integrated into the rest. Next, the whole is tested in what Constantine (2002) calls “regression testing,” until the new feature works with all the previously developed pieces. As a result, the next iteration always begins with something that works.

The agile software development methods are further characterized by a profound emphasis on communication, especially continuous communication with the customer. Informal communication is strongly preferred over formal. Close communication is emphasized to the point that they have an onsite customer as part of the team, giving feedback continuously.

26.2.2.3 Avoiding big design upfront

A main principle of agile SE methods is to avoid big design upfront (BDUF). This means the approach generally eschews upfront field studies and extensive requirements engineering. The idea is to get code written as soon as possible and resolve problems by reacting to customer feedback later.

SE practitioners verify that they are writing the code correctly by the practice of pair programming. Code is written by two programmers working together and sharing one computer and one screen; that is, a colleague is always watching over each programmer's shoulder.

Of course, pair programming is not new with agile methods. Even outside agile SE methods (and before they existed), pair programming was a proven technique with a solid track record (Constantine, 2002). Code is further verified by regular and continuous testing against an inventory of test cases.

26.3 LIFECYCLE ASPECTS OF AGILE SOFTWARE ENGINEERING

If the agile SE process were to be represented by a lifecycle diagram, then it would not be a waterfall or even an iteration of stages, but a set of overlapping micro-development activities. In agile approaches, developers do just enough of each activity to support one small feature, as shown on the right-hand side of Fig. 26.2.

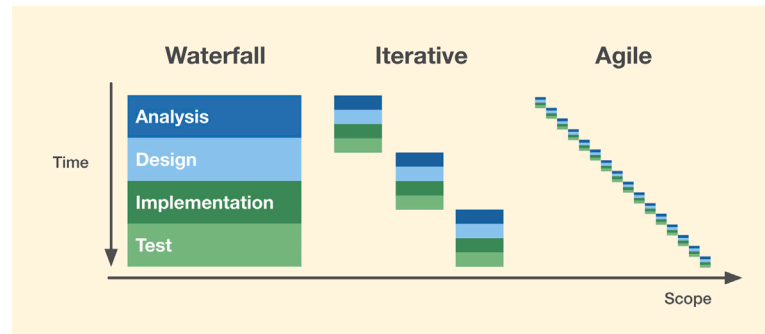


Fig. 26.2
Comparison of scope of development activities across methodologies (adapted from Beck [1999], permission of IEEE).

This contrast shows that building an e-commerce website in the waterfall approach would require analysis (listing all requirements that must be supported in the website) before starting top-down design. In an agile approach, you would forego requirements gathering and build the same website as a series of smaller features, such as a search function, shopping cart, and then a checkout module.

26.3.1 Planning in Agile Software Engineering Methods

In our discussion of how an agile SE method works,³ we are roughly following XP as a guide. As shown in Fig. 26.3, each iteration consists of two parts: planning and a sprint to implement and test the code for one release. A sprint takes place in a relatively short (<1 month) time period in which a working and releasable product increment is implemented. It is a unit of work being done in an agile SE environment; this iteration is associated with a release, a mini-software product that the client and/or users can exercise and explore.

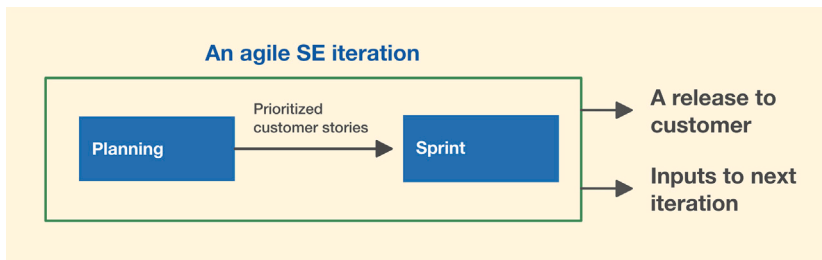


Fig. 26.3

Abstraction of an agile software engineering (SE) release iteration.

26.3.1.1 Customer stories

We have already discussed user stories (see Chapter 10). Now, we show how they play a pivotal role in connecting the UX and SE in late funnel development activities.

For the moment, we will use customer stories, which have been used in the context of agile SE because they do not work with users (customers are clients who pay to build the system; users are those who would use that system). As we bring agile UX into this process, we will be switching to user stories, which can include both sources.

The planning part of each SE iteration in Fig. 26.3 yields a set of customer-written stories, prioritized by business value and cost to implement. A customer story is a bit like a use case, scenario, or requirement. A customer story, written on a story (index) card, is a description of a customer-requested feature. It is a narrative about how the system is supposed to solve a problem, representing a chunk of functionality that is coherent and useful in some way to the customer.

³This is the way things should work in theory. As with most UX and SE methods, there are many variations in practice.

26.3.1.2 Story-based planning

Expanding the planning box of Fig. 26.3, we get the details of how customer stories are used in SE planning (Fig. 26.4).

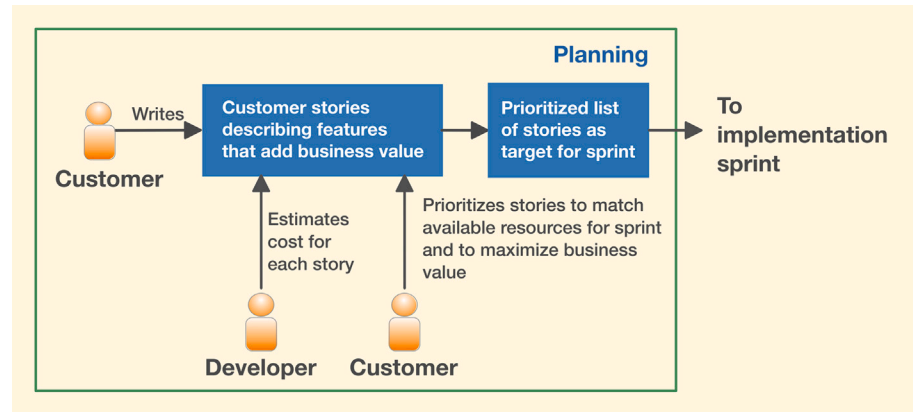


Fig. 26.4
Customer stories as the basis
of agile software engineering
planning.

As shown in Fig. 26.4, developers begin the planning process by meeting with onsite customer representatives. They ask customer representatives to think about the most useful chunks of functionality that can add business or enterprise value. The customer writes stories about the need for these pieces of functionality. This is the primary way that the developers understand requirements through the customer representatives.

Developers assess the stories and estimate the effort required to implement (i.e., program) a solution for each, writing the estimate on the story card. Typically in XP, each story gets a 1- to 3-week estimate in ideal development time. (More on how these estimates are created follows.)

The customer sorts and prioritizes the story cards by choosing a small set for which the cost estimates are within a predetermined budget and represent features they want to include in a release. Prioritization may result in lists of stories or requirements labeled “do first,” “desired—do, if time,” and “deferred—consider next time.” Developers break down the stories into development tasks, each written on a task card (for the developers to perform). Each such development task is given a point count representing an estimate of the amount of development effort required to implement that task. Then, all tasks in a story card are totaled to arrive at the total ideal development time.

The output of the planning box, which goes to the upcoming implementation sprint, is a set of customer-written stories, prioritized by cost to implement.

26.3.1.3 Managing customer stories and development tasks

The most popular project management system used in industry practice to manage stories and associated tasks is Atlassian's JIRA (from the Japanese word *Gojira*, which means "Godzilla") software.⁴ This system provides product managers and SE roles the capabilities to plan releases, manage user stories and associated tasks for current and future sprints, create acceptance criteria for each user story, and know the current status of each story. JIRA also provides a robust defect (bug) tracking capability, including the ability to plan which bugs will be fixed in a given release, including dependencies and other barriers to resolving those bugs.

26.3.2 Controlling Scope

Customer stories are the local currency in what Beck (2000, p. 54) calls the "planning game," through which the customer and the developers negotiate the scope of each release. At the beginning, there is a time and effort budget of the person-hours or level of effort available for implementing all the stories, usually per release.

As the customer prioritizes story cards, the total of the work estimates is kept, and when it reaches the budget limit, the developers' dance card is full. Later, if the customer wants to cut in with another story, the customer has to decide which existing story with an equal or greater work estimate must be removed to make room for the new one. So, no one, not even the boss, can add more features.

This approach gives the customer control of which stories will be implemented, but affords developers a tool to battle scope or feature creep. Developer estimates of effort can be way off, probably in most cases underestimating the effort necessary, but at least it lets them draw a line in the sand. With experience, developers get pretty good at this estimation given a particular technology platform and application domain. The better the developers get at estimation, the more smoothly the entire project will go.

26.3.3 Sprints in Agile Software Engineering Methods

Fig. 26.5 shows an expansion of the sprint box in Fig. 26.3. Each agile SE sprint consists of activities that are described in the following sections.

⁴<https://www.atlassian.com/software/jira>

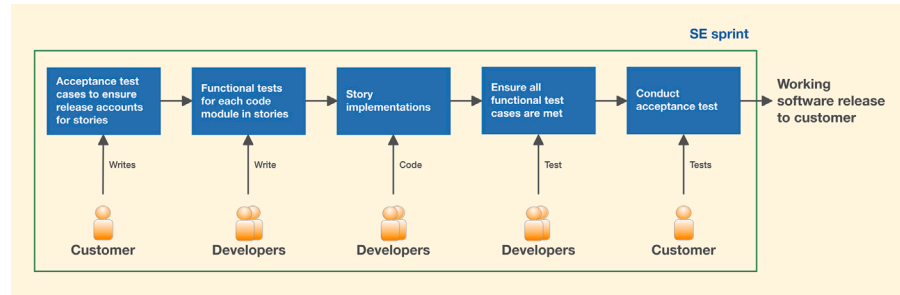


Fig. 26.5

An agile software engineering (SE) sprint.

26.3.3.1 Acceptance test creation

The customer writes the functional acceptance tests. There is no process for this, so it may be fuzzy, but it puts the customer in control of acceptance of the eventual code. With experience, customers get good at this. Usually the acceptance criteria, the basis for acceptance tests, are summarized in the corresponding customer stories as a list of feature capabilities that the customer has to perform.

26.3.3.2 Unit code test creation

The team divides the work by assigning customer stories to be coded in the next sprint. A programmer picks a customer story card and finds a programming partner. Together, before any coding, the pair writes unit tests that can verify functionality is present in the code yet to be written.

26.3.3.3 Implementation coding

The programming pairs work together to write the code for modules that support the functionality of selected customer stories. As they work, the partners do on-the-fly design. The agile SE literature says almost nothing about design.

The programmers do not worry about the higher-level architecture; the system architecture supposedly evolves with each new slice of functionality added to the overall system. The programming pair integrates this code into the latest version.

26.3.3.4 Code testing

Next, the programming pair runs the unit code tests designed for the modules just implemented to ensure that the realization of this functionality in code is correct.

26.3.3.5 Regression testing

In addition to testing the code for the current features, the team runs all code tests again on all modules coded thus far until all tests are passed, to ensure that the new module does not break any of the previously implemented modules. This allows developers to make code modifications based on changing requirements while ensuring that all existing parts of the code continue to function properly.

26.3.3.6 Acceptance testing and deployment

Developers submit this potentially shippable product functionality to the customer for acceptance review. Upon acceptance, the team deploys this small iterative release to the customer.

26.4 CHALLENGES OF AGILE SOFTWARE ENGINEERING METHODS FROM THE UX PERSPECTIVE

Agile methods make SE practitioners feel productive and in control because they and their customers, not the overarching design, drive the process. These methods are inexpensive, fast, and lightweight, with early deliverables. The pair-programming aspect also seems to produce high-reliability code with fewer bugs. Nonetheless, agile SE methods are programming methods developed by programmers for programmers, and they pose some challenges from the UX perspective.

Because BDUF is shunned in an agile SE method, there is no upfront analysis to establish a conceptual view of the system and the users' work practice. The one customer representative on the team is not required to be a real user and cannot be expected to represent all viewpoints, needs and requirements, usage issues, or usage context. There may be no real user data upfront, and coding will end up being "based only on assumptions about user needs" (Mommel, Gundelsweiler, & Reiterer, 2007, p. 169). There is no identification of user tasks and no process for identifying user interaction or information needs.

Beyer, Holtzblatt, and Baker (2004) echo this criticism of using a customer representative as the only application domain expert. As they point out in their second axiom, "Make the user the expert," many customer representatives are not also users and therefore cannot speak for the work practice of others. Beyond these predesign drawbacks, the agile SE method does not involve UX design, so there is no room in the process for design ideation.

26.5 WHAT IS NEEDED ON THE UX SIDE

Nonetheless, in some ways, the UX process lifecycle is a good candidate to fit with agile software methods because it is already iterative. But there is a big difference. The traditional UX lifecycle is built on starting with a complete understanding of users and their needs followed by extensive design activities, both missing in agile SE. Regarding the early agile UX research, Beyer, Holtzblatt, and Baker (2004) recommend devoting an iteration of UX research with real users to understand user needs. They report getting through quick UX research and early design with five to eight users in 1 to 2 weeks. Our experience, even on medium-complexity projects, has been closer to 1 to 2 months for an unfamiliar work domain.

In this section, we discuss the considerations necessary to adjust traditional UX methods to adapt to agile SE approaches (Mammel, Gundelsweiler, & Reiterer, 2007). To work in the UX domain, an agile method must involve some early analysis devoted to understanding user work activities and work context and gleaning general concepts of the system within its ecology. Some early design activities must also be retained in the process to give structure and coherence to how interaction and information fit into the design. If top-down design is needed for an innovative product design, then it should take place before any involvement on the SE side as well. This can be difficult to arrange, especially in projects where the SE people have all the control, but early-funnel (for large-scope activity, usually for conceptual design, before syncing with software engineering [see [Section 4.4.4](#)]) UX activities are supposed to accommodate exactly those needs.

At the same time, to be compatible with the agile SE side of development, UX methods must:

- Be lightweight
- Emphasize team collaboration and require colocation
- Include effective customer and user representatives
- Adjust UX lifecycle activity timing to be compatible with SE sprint-based incremental releases by switching focus from top-down holistic design to bottom-up feature design
- Include ways to control scope

Late-funnel UX activities are intended to take care of these needs.

26.6 PROBLEMS TO ANTICIPATE

In a special-interest-group workshop at CHI 2009 (Miller & Sy, 2009), a group of UX practitioners met to share their experiences in trying to incorporate a user-centered design approach into the agile SE process. Among the difficulties experienced by these practitioners in their own environments were:

- Sprints too short; not enough time for customer contact, design, and evaluation
- Inadequate opportunities for user feedback, and the user feedback they did get was ignored
- Customer representative weak, not committed
- Lack of colocation makes it difficult to collaborate closely
- No shared vision of broader conceptual design because focus is on details
- Risk of piecemeal results in a purely bottom-up approach

26.6.1 UX and SE May Not Work Well Together

In Fig. 26.6, we show a general picture of how UX and SE are supposed to work together in design and implementation.

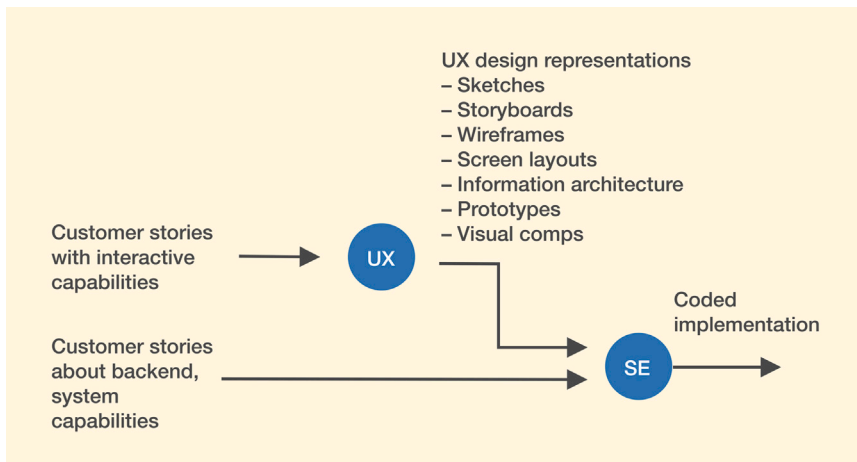


Fig. 26.6
How UX and software engineering (SE) work together.

In either the traditional or agile lifecycle, the UX team is supposed to provide a UX design as specifications to the SE team to implement and integrate with the corresponding functionality. If everyone on both sides of the project (and management as well) agrees to this simple model at the beginning, the matter of integrating UX and SE within the project will be well on its way to being solved.

In reality, however, the SE team is often well into its process when the UX team is introduced to the project, and the UX team is doomed to play

catch-up for the rest of the project, and the project can never benefit from what the UX team could have done. In the worst of scenarios, the UX team is called in to review and give feedback on the work done so far by the SE team.

In practice, the customer stories can be classified into two buckets: those that describe product capabilities with interactive capabilities and those that are purely about non-interactive system or software issues, such as security or database design. While it looks like the UX team can focus on the former type of customer stories, generate design specifications from them and pass them along to the SE team, and the SE team can just focus on the customer stories about the backend system, in practice, it is important for all this to be done in a collaborative and open way. The main requirement is that all inputs obtained from the client or users are shared with both teams. The SE team must agree to not gather requirements or design recommendations without involving the UX team from the beginning.

26.6.2 The Need for a Full Overview: The Software Side Versus the UX Side

Because the agile approach to software engineering is incremental and bottom-up, it precludes the luxury of seeing a full overview of the system upfront. In fact, some agile teams see not having an overview as a “badge” testifying to their correct use of the agile method—that is, not being distracted from their feature-at-a-time emerging view of the system. The whole point of the agile approach is that they cannot afford to establish a system overview upfront, nor do they need to because the user does not see the software directly.

Building a system a little piece at a time can be risky, however, at least on the UX side. The user’s view of the system is the user interface. This view, depending directly on the UX design, must look as though it is built around a well-integrated conceptual model, a single unified style or theme, not a patchwork of different ideas and views. If not, it will lead to a confusing and disjointed user experience. It is almost impossible to develop a unified user interface exclusively by piecewise and bottom-up methods, one feature at a time. So, the UX team needs an overview to emerge as soon as possible.

26.7 A SYNTHESIZED APPROACH TO INTEGRATING AGILE UX AND AGILE SE

Most of the early literature about incorporating UX into the agile SE development process is based on either adjusting UX design methods to somehow keep pace with existing agile SE methods or trying to do selected parts of UX design processes in the presence of an inflexible agile SE method.

Although it is possible that XP, for example, and some abbreviated UX design techniques can coexist and work together in these add-on approaches, the two parts do not combine well (McInerney & Maurer, 2005; Patton, 2002, 2008). This creates a coping scenario on the UX side, as UX practitioners attempt to live with the constraints while trying to ply their own processes within an overall development environment driven solely by the agile SE method.

Here, we have tried to synthesize an approach to allow our agile UX process to mesh with an agile SE environment without compromising on essential UX needs. We especially acknowledge the influence of Beyer, Holtzblatt, and Baker (2004), Constantine and Lockwood (2003), Meads (2010), and Miller (2005).

26.7.1 Integrating UX into Planning

Fig. 26.7 shows a scheme for integrating the UX role into the planning box of Fig. 26.4.

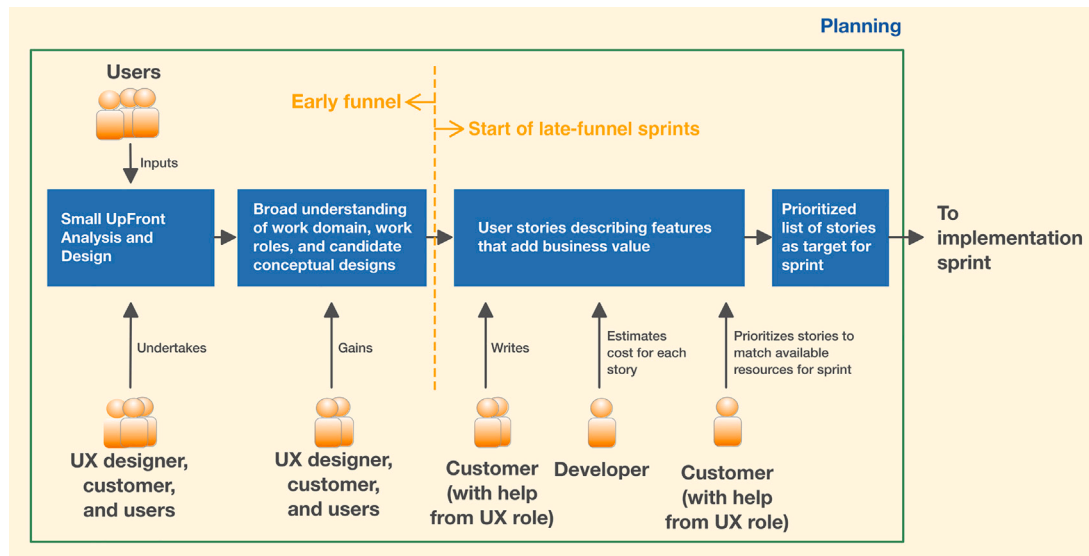


Fig. 26.7
Integrating the UX role
into early agile UX funnel
planning.

Planning in Fig. 26.7 features small upfront analysis and design by the UX designer, customer, and users, in the left-most box, producing a broad (if partial) understanding of the work domain, work practice, and conceptual model (next to left-most box). This upfront UX activity occurs in the early agile UX funnel.

26.7.1.1 *Small upfront analysis and design*

Early funnel work. In [Section 26.2.2.3](#), we described how SE practitioners avoid big design upfront, preferring instead to write *some* code as soon as possible and then fix problems by reacting to customer feedback later. In UX, we cannot afford to operate this way. The integrity and consistency of all our designs, including the small-scope task-level designs of the late funnel, depend on first establishing a solid (even if not comprehensive) overall conceptual design in the early funnel. The necessary groundwork cannot turn the whole process into a waterfall model, however, so it has to be light on its feet, which is where the small upfront analysis and design come in.

The small upfront analysis and design, in the left-most box of [Fig. 26.7](#), is our way of including an initial abbreviated form of UX research and design that involves the customer and users. This is exactly what happens in the early agile UX funnel. Much of the design work in the early funnel small upfront analysis and design is conceptual design to establish a consistent high-level overview.

In planning, the UX person also can offer the customer valuable assistance in writing and prioritizing stories. These stories are now called user stories rather than customer stories because their substance came from users in the upfront analysis. Although this begins to change the basic agile pattern, it gives the UX team more traction in bringing UX into the overall process. Other authors (Constantine & Lockwood, 1999; Memmel, Gundelsweiler, & Reiterer, 2007) have agreed that adding a measure of user and/or task modeling is also a very useful supplement in that same spirit.

Goals of the small upfront analysis and design in the early agile UX funnel include:

- Understand the user's work and its context
- Identify key work roles, work activities, and user tasks
- Model workflow and activities in the existing enterprise and system
- Forge an initial high-level conceptual design
- Identify inputs for selected user stories that reflect user needs in the context of the work practice

These early agile UX funnel small upfront analysis and design activities are an abbreviated version of what we describe in Part 2 of this book.

Data capture. Agile UX research can be as brief or as lengthy as desired or afforded. We suggest interviewing and observing the work practice of at least one or two people in each key work role. The UX practitioners write notes by hand directly on small index cards—a Constantine hallmark—to discourage verbosity in the notes.

Aim toward effective user stories. We are looking for user stories to drive our small pieces of interaction design and prototyping, but what kind of user stories do we seek? Stories about work activities, roles, and tasks can still be a good way for designers to begin; however, as Meads (2010) says, users are not interested in tasks per se, but in features. Following his advice, we focus on features, which can encompass a substantial set of related user work activities and tasks within a work context. Therefore we usually develop a set of related user stories for a feature (e.g., tasks to create, search for, display, modify, and delete subscriptions to a certain kind of service).

26.7.1.2 UX role helps customer write user stories

In the third box from the left in [Fig. 26.7](#), the UX person helps the customer write user stories. As we said earlier, the UX person usually takes charge of user story writing to ensure completeness, appropriate scope, consistency, and cohesion.

Because UX team members, customers, and users collaborate in the early agile UX funnel small upfront analysis and design, user story writing will be easier, faster, and more representative of real user needs. The UX role influences the customer toward creating stories based on workflows revealed in the agile UX research part of small upfront analysis and design.

26.7.1.3 The truth about user stories

As we explained in [Section 10.2.1.1](#), you cannot depend on stories written only by users to be accurate, complete, or comprehensive. In real projects, UX analysts and designers take inputs from users collected in UX research and write small-scope user capability requirements as user stories, as though users had expressed them, so the whole collection can be more complete and expressed in a uniform format.

26.7.1.4 UX role helps customer prioritize user stories

By helping the customer representative prioritize the user stories, the UX person can keep an eye on the overarching vision of user experience and a cohesive conceptual design, thereby steering the result toward an effective set of stories for an iteration.

26.7.2 Integrating UX into Sprints

In [Fig. 26.8](#), we show UX counterpart activities occurring in the late funnel during an agile SE sprint of [Fig. 26.5](#).

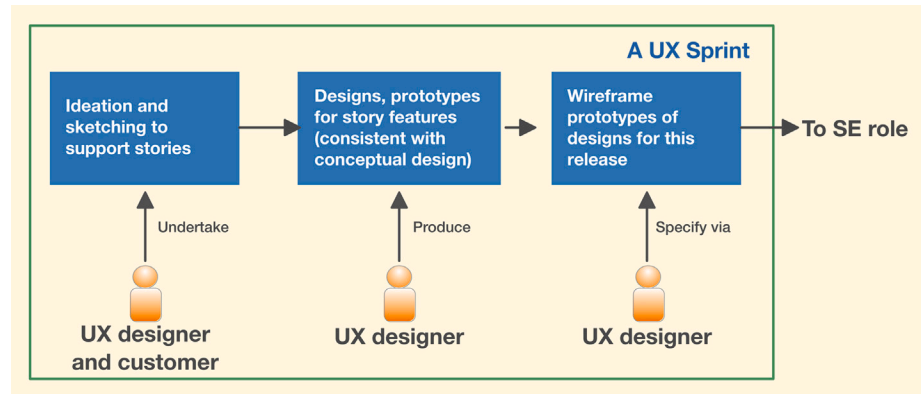


Fig. 26.8

UX counterpart of an agile software engineering (SE) sprint.

While the SE people are doing a sprint, the UX person and customer perform their own version of a sprint (see Fig. 26.8). They begin by picking a story or picking a feature with a set of related user stories.

With the conceptual design in mind, the UX designers begin ideation and sketching of an interaction design to support the functionality of the user story. Initial static wireframe prototypes are used as sketches of interaction designs, leading to interactive click-through wireframe prototypes to demonstrate navigation within interaction sequences.

Design walk-throughs and iterative design fixes. At this small scope, the tightly coupled design creation–prototyping–evaluation cycle will often begin with frequent multiple static wireframe sketches. These sketches are iteratively reviewed within the UX team for ideation in task-level design creation. This early design ideation and sketching can also involve brief focused returns to UX research to fill gaps and answer questions about the feature.

When the UX team is satisfied they have their best initial design, it is demonstrated in the form of a click-through wireframe prototype in a walk-through review with all stakeholders, including client/customers and users. The typical result is possibly multiple rounds of feedback, refinement, and further evaluation.

Then, the same kind of walk-through evaluation is done with SE developers to get feedback on feasibility, consistency, platform issues, etc. This may lead to multiple rounds of fixing and refining, leading to a handoff to developers for implementation. By this time, UX designers and software developers should be on the same page about the feature.

After implementation, the UX designer should check the result against the UX designs for fidelity of implementation. Then, the whole

team should submit the working feature to the customer for acceptance review.

Table 26.1 shows a mini-UX checklist developed for this process while Hartson was working as a consultant at the Network Infrastructure and Services group at Virginia Tech.

Table 26.1

Late-funnel mini-UX process checklist

Activity step	Date performed	Person(s)
View user stories in JIRA (Section 26.3.1.3)		
Task-level UX research and modeling: identify work roles, hierarchical task inventory context, task sequences, business rules, relevant existing apps/screens, workflow		
As needed: define terminology, database schemas		
Ideation and sketching of initial UX design		
Draw state diagram of flow, if appropriate		
Draw wireframes in Sketch		
Connect wireframes with navigation links in InVision and Craft		
Create JIRA user story (if SE did not create it yet) and add link to wireframe prototype		
With UX group: design walk-through, iterate		
Get UX agreement		
With SE: initial (pre-implementation) design walk-through		
Update wireframe prototype with changes from SE		
Increase prototype fidelity, if useful		
With non-UX stakeholders: design walk-through, iterate as needed.		
With SE: design walk-through and handoff		
UX: update JIRA issue, add link to InVision, assign issue to SE		
SE: update JIRA issue, assign issue to UX (dependency link to original issue)		
QA: implementation fidelity check (UX responsibility)—what is built should reflect wireframe prototype exactly		
If no discrepancy, sign off (UX) and close JIRA issue		
Else: UX create JIRA issued "update software with UX review feedback"		
Repeat until no discrepancy		

26.7.3 Synchronizing the Two Agile Workflows

After describing agile SE planning, agile SE sprints, and UX integration in the late funnel, we now talk about how the UX and SE teams work together and synchronize the workflow in their respective parts of the agile process.

26.7.3.1 Dovetailed work activities

Miller (2005) proposed a “staggered” approach to parallel track agile development that featured a crisscrossed interplay between UX activities and SE activities across multiple cycles of agile development. As Patton (2008) stated in his blog, the overall approach is characterized as “work ahead, follow behind.”

As Patton (2008) says, UX people on agile teams “become masters of development time travel, nimbly moving back and forth through past, present, and future development work.” Based roughly on Miller’s (2005) idea, and adding our early-funnel concept, we show a scheme in [Fig. 26.9](#) for how UX people and SE people can synchronize their work via a dovetail alternation of activities across progressive iterations.

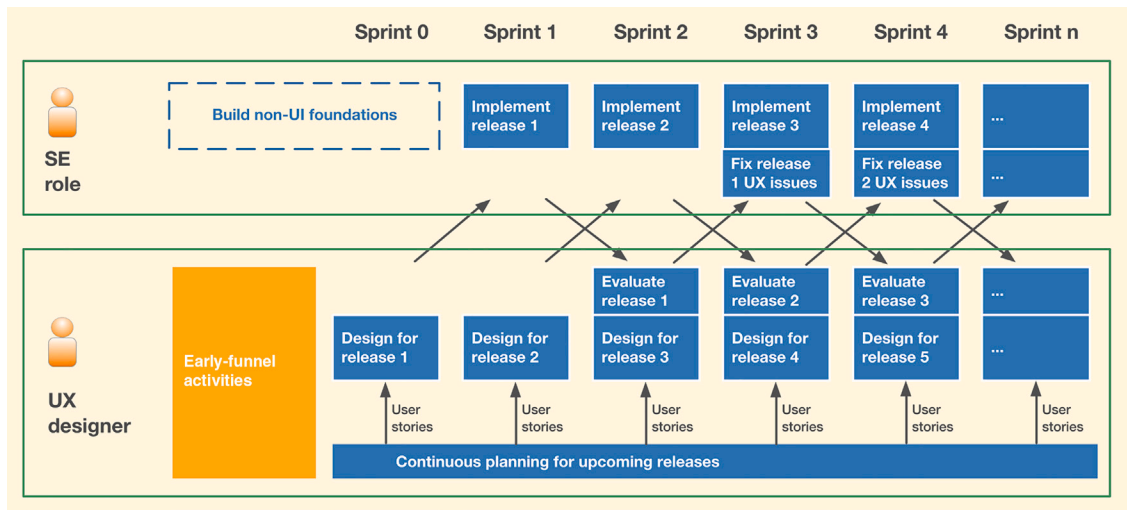


Fig. 26.9
Alternating UX and
software engineering (SE)
workflow in an agile process.

In the original agile SE approach, SE people began first with sprint 1, taking a set of stories and building a release. That worked when the only thing that was happening was implementation. Now that we are bringing in UX

design and evaluation into the mix, we need a few changes, beginning with the early funnel activities before going on to initial sprint 0 on the UX side.

Sprint 0 follows early funnel work on UX research and conceptual design on the UX side and is for beginning the synchronization with the SE cadence. The amount of time these early funnel activities will take depends on the size and complexity of the product or system and the familiarity of the UX team with the problem domain. At the end of sprint 0, the UX team should have UX designs for the initial feature ready to give to the SE people for their sprint 1. During this time, the SE people focus on other things, such as building the software infrastructure and services required to support the whole system, what Miller calls building the “high-development, low-UI features.” When the UX people finish their designs for release 1, they hand them off to SE people for implementation in sprint 1, which includes implementation of both functional stories and interaction design components for that cycle.

Not all UX design challenges are equal; sometimes, there is not enough time to address UX design adequately in a given sprint. Then, that design will have to evolve over the design and evaluation activities of more than one sprint. Also, sometimes in interaction design, we want to try out two variations because we are not sure, so that will have to take place over multiple sprints.

Because of the staggering or dovetailing of activities, people on each part of the team are typically working on multiple things at a time. For example, right after handing off the designs for release 1 to the SE people, the UX people start on designs for release 2 and continue doing this until the end of sprint 1 (while the SE people are coding release 1). In any given sprint, say, sprint n , the UX people are performing UX research and planning for sprint $n + 2$, while doing design (and prototyping) for sprint $n + 1$, and evaluation of the design for sprint $n - 1$.

Following the lifecycle of a single release, release n , we see that in sprint $n - 1$, the UX role designs for release n , to be implemented by the SE people in sprint n . UX evaluates release n in sprint $n + 1$. SE fixes it in sprint $n + 2$ and re-releases it at the end of that sprint.

26.7.3.2 *The value of early delivery*

As Memmel, Gundelsweiler, and Reiterer (2007) say, you have the potential to deliver design visions extremely early in the process, resulting in amazingly early involvement of the customer.

Feedback for the first feature can go far beyond that one feature and its usage. This is the first opportunity for the team to get any real feedback.

Many additional things can come out of that, not specific to that feature. For example, you will get feedback on how the process is working. You will get feedback on the overall style of your design. You will hear other questions and issues that customers are thinking about that you would not have access to until much later in a non-agile process. Your customers may even reprioritize the story cards based on this interaction. This early feedback fits well with the agile principles of abundant communication and the expectation of early and frequent change.

26.7.3.3 *Continuous delivery*

Delivery to customers and users is continuous but incremental. At the end of any given sprint (call it sprint n), the customer sees a UX prototype of the upcoming release; in the next sprint, sprint $n + 1$, the customer sees the full functional implementation of that prototype. In sprint $n + 2$, the customer sees UX evaluation findings for that same prototype; in sprint $n + 3$, the customer sees the final redesign. Each of these points in time is an opportunity for the customer to give feedback on the interaction design.

26.7.3.4 *The importance of regression testing*

Regression testing is a step in the agile SE process used to integrate the most recent tested-and-passed feature with all those that came before it. Without regression testing, the agile method is little more than an incremental waterfall method. You are stuck with decisions made in sprints 0 and 1, with no ability to iterate major concepts. Unfortunately, time pressures of sprints in real practice can cause teams to shortcut regression testing.

However, this kind of quick testing can be easy, and for the most part automated on the SE side. On the UX side, it is a whole different story. Doing regression testing on the UX side would entail running an evaluation after each sprint and using all measuring instruments (e.g., benchmark tasks or surveys) from all previous sprints plus those for the current sprint before releasing it to the customer. In practice, there is almost never enough time to do this.

26.7.3.5 *Planning across iterations*

Fig. 26.9 shows planning in a single box at the bottom, continuously extending across all sprint cycles. That is to convey the idea that planning does not occur in discrete little boxes over time at a specific spot in the flow. Planning is more of an overarching activity, distributed over time, and it is cumulative in that

the process builds up a knowledge base founded on agile UX research. The planning process does not start over for the planning of each cycle.

Instead, the same knowledge base is consulted, updated, and massaged, working with the original small upfront analysis and design results and anything added to supplement those results. Because an overview and conceptual design are evolving in the process, this kind of UX planning brings some top-down benefits to an otherwise exclusively bottom-up process.

26.7.3.6 Communication during synchronization

This kind of interwoven development process brings with it the risk of falling apart if anything goes wrong. This intensifies the need for constant communication so that everyone remains aware of what everyone else is doing, what progress is being made by others, and what problems are being encountered.

Agile processes can be more fragile than their heavyweight counterparts. Because each part depends on the others in a tightly orchestrated overall activity, if something goes wrong in one place, then there may be little time to react to surprises. Constant communication can help minimize this risk.

26.8 SEGUE

This concludes Part 6 and our discussion of process concepts in this book. Next, in Part 7, we get into supporting UX design with the theory of UX affordances and UX design guidelines.